

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación gráfica, visión computacional y multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Clasificación y Reconocimiento</i>				
NÚMERO DE PRÁCTICA:	<i>09</i>	AÑO LECTIVO:	<i>2026A</i>	NRO. SEMESTRE:	<i>IX</i>
FECHA DE PRESENTACIÓN	<i>08/07/2026</i>	HORA DE PRESENTACIÓN			
INTEGRANTE (s): Maxs Sebastian Joaquin Forocca Mamani				NOTA:	
DOCENTE(s): <i>RENE ALONSO NIETO VALENCIA</i>					

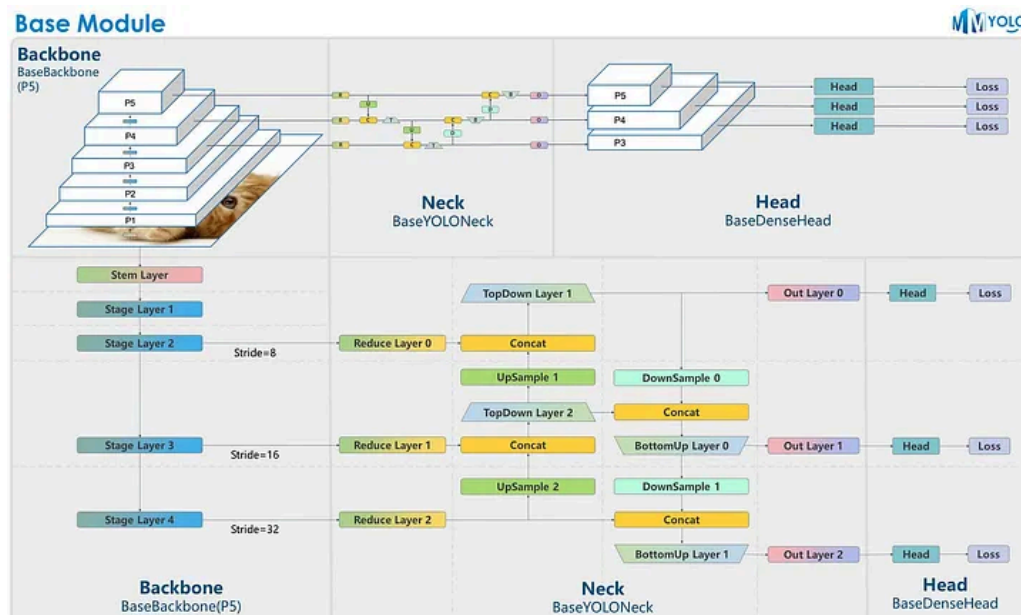
SOLUCIÓN Y RESULTADOS	
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Descripción General del Programa El presente proyecto implementa un sistema dual para la clasificación y reconocimiento de objetos en imágenes mediante técnicas modernas de visión artificial. El sistema aborda dos enfoques prácticos: el primero ejecuta la detección y segmentación en tiempo real utilizando la cámara web del dispositivo, mientras que el segundo ofrece una interfaz gráfica web intuitiva que permite a los usuarios cargar imágenes estáticas desde su almacenamiento local para ser procesadas bajo demanda. La temática del software es de carácter generalista, aprovechando el potencial de un modelo preentrenado capaz de identificar de manera inmediata hasta 80 clases diferentes de objetos de la vida cotidiana. 2. Marco Teorico 2.1. YOLO YOLO (You Only Look Once) es una arquitectura de red neuronal convolucional (CNN) profunda que revolucionó el campo de la visión computacional. A diferencia de los enfoques tradicionales que proponen múltiples regiones de interés en una imagen y luego las clasifican de forma independiente, YOLO plantea la detección como un único problema de regresión. Divide la imagen en una rejilla y predice simultáneamente las	

cajas delimitadoras (bounding boxes) y las probabilidades de clase de extremo a extremo en una sola pasada hacia adelante por la red, lo que permite un rendimiento en tiempo real insuperable.

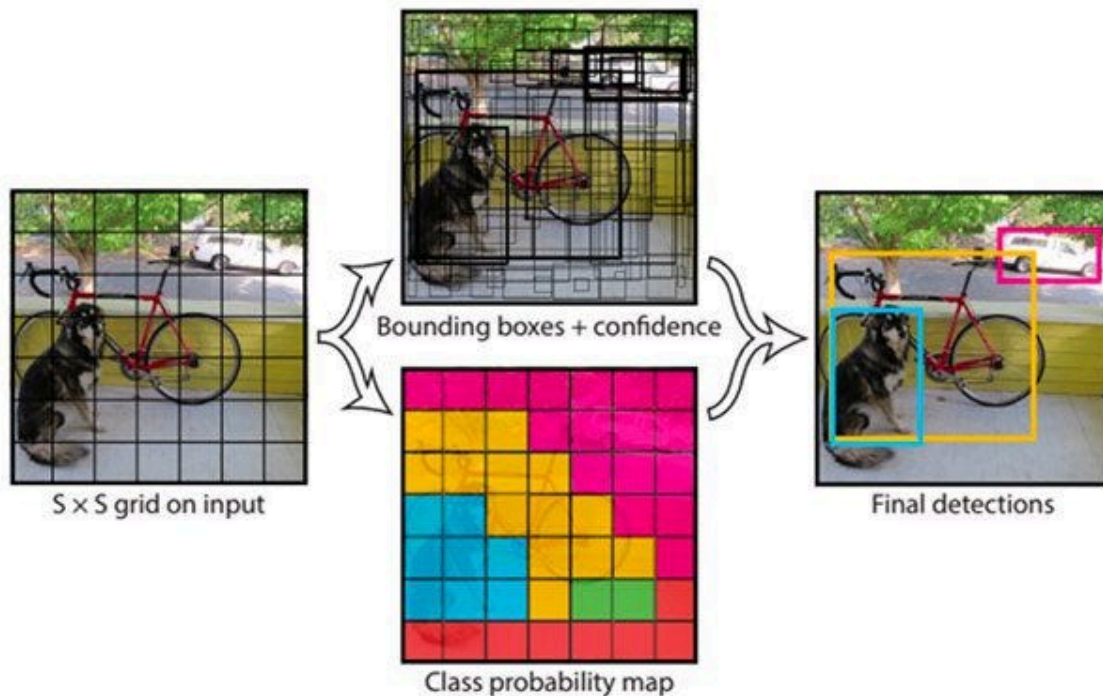
2.2. Evolucion y Características de YOLOv8

YOLOv8, desarrollado por Ultralytics, representa el estado del arte de esta familia de modelos. Introduce mejoras sustanciales respecto a sus predecesores:

- **Arquitectura sin Anclas (Anchor-free):** A diferencia de las versiones previas que usaban cajas de anclaje predefinidas, YOLOv8 predice directamente el centro de los objetos y la distancia a los bordes. Esto reduce drásticamente el número de parámetros de la red, agiliza el entrenamiento y mitiga errores en objetos de escalas inusuales.
- **Nueva Red de Características (Backbone):** Utiliza una variante mejorada de la arquitectura CSP (Cross Stage Partial) llamada C2f, la cual optimiza el flujo de gradientes, enriqueciendo la extracción de características complejas sin comprometer la velocidad de procesamiento.



- **Segmentación y Clasificación Nativa:** Está diseñado desde cero no solo para la detección clásica de cajas, sino también para tareas de segmentación de instancias y estimación de posturas corporales (pose estimation).



3. Requerimientos y Tecnologías Utilizadas

Para garantizar la reproducibilidad del entorno de desarrollo, se configuró el proyecto bajo el lenguaje de programación Python utilizando las siguientes versiones específicas de dependencias (requirements.txt):

- opencv-python==4.10.0.84: Librería fundamental para la adquisición digital de imágenes, manejo de flujos de video e interacción básica de ventanas.
- numpy==1.26.4: Soporte para el manejo eficiente de matrices multidimensionales, estructura esencial sobre la cual se representan digitalmente los píxeles de los fotogramas.
- ultralytics==8.4.0: Framework oficial que provee la arquitectura y los pesos del modelo preentrenado YOLOv8 (yolov8n.pt).
- cvzone==2.0.0: Librería de alto nivel construida sobre OpenCV que optimiza el renderizado gráfico de textos, esquinas decorativas y FPS.
- streamlit==1.59.0: Entorno de desarrollo ágil que transforma scripts de Python en aplicaciones web interactivas sin necesidad de codificar HTML, CSS o JavaScript.
- tensorflow==2.16.1: Backend de aprendizaje profundo utilizado para dar soporte y compatibilidad a operaciones de tensores en arquitecturas de IA.

4. Explicación Detallada del Funcionamiento del Código

4.1. Script 1: Detección en Tiempo Real mediante Cámara Web

Este script se encarga de abrir un canal de comunicación directo con la cámara web del computador, procesando cada cuadro por segundo (FPS) recibido para detectar objetos dinámicos.

Python

```
import cv2
from ultralytics import YOLO
import cvzone

# 1. ADQUISICIÓN: Se inicializa la captura de video de la cámara por defecto (ID 0)
cap = cv2.VideoCapture(0)
cap.set(3, 1280) # Configura la resolución horizontal (Ancho)
cap.set(4, 720) # Configura la resolución vertical (Alto)

# Carga el modelo preentrenado YOLOv8 en su versión Nano (ligera y veloz)
model = YOLO("yolov8n.pt")

# Bucle infinito para procesar el flujo continuo de video analógico a digital
while True:
    success, img = cap.read()
    if not success:
        break # Rompe el ciclo si hay fallas en la captura de la señal

    # 2. SEGMENTACIÓN E INTERPRETACIÓN: El modelo procesa el fotograma
    # 'stream=True' activa el uso de generadores en Python para optimizar la memoria RAM
    results = model(img, stream=True)

    # Iteración sobre los resultados de detección analizados por la red
    for r in results:
        boxes = r.boxes
        for box in boxes:
            # Extracción geométrica de las coordenadas de la caja delimitadora (Bounding
            Box)
            x1, y1, x2, y2 = box.xyxy[0]
            x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

            # Obtención del porcentaje de certeza / confianza matemática del modelo
            conf = round(float(box.conf[0]) * 100, 2)

            # Mapeo del índice numérico al nombre de la clase correspondiente (base de
            conocimientos)
            cls = int(box.cls[0])
```

```
name = model.names[cls]

# Filtro de confianza para mitigar la aparición de falsos positivos en el
escenario virtual
if conf > 50:
    # DIBUJO DE RESULTADOS: Renderiza esquinas de diseño estilizado alrededor
del objeto
    cvzone.cornerRect(img, (x1, y1, x2 - x1, y2 - y1), l=15, rt=2, colorR=(0,
255, 0))
    # Superpone una tarjeta con el nombre y la confianza calculada
    cvzone.putTextRect(img, f'{name} {conf}%', (max(0, x1), max(35, y1)),
scale=1, thickness=1, colorR=(0, 255, 0))

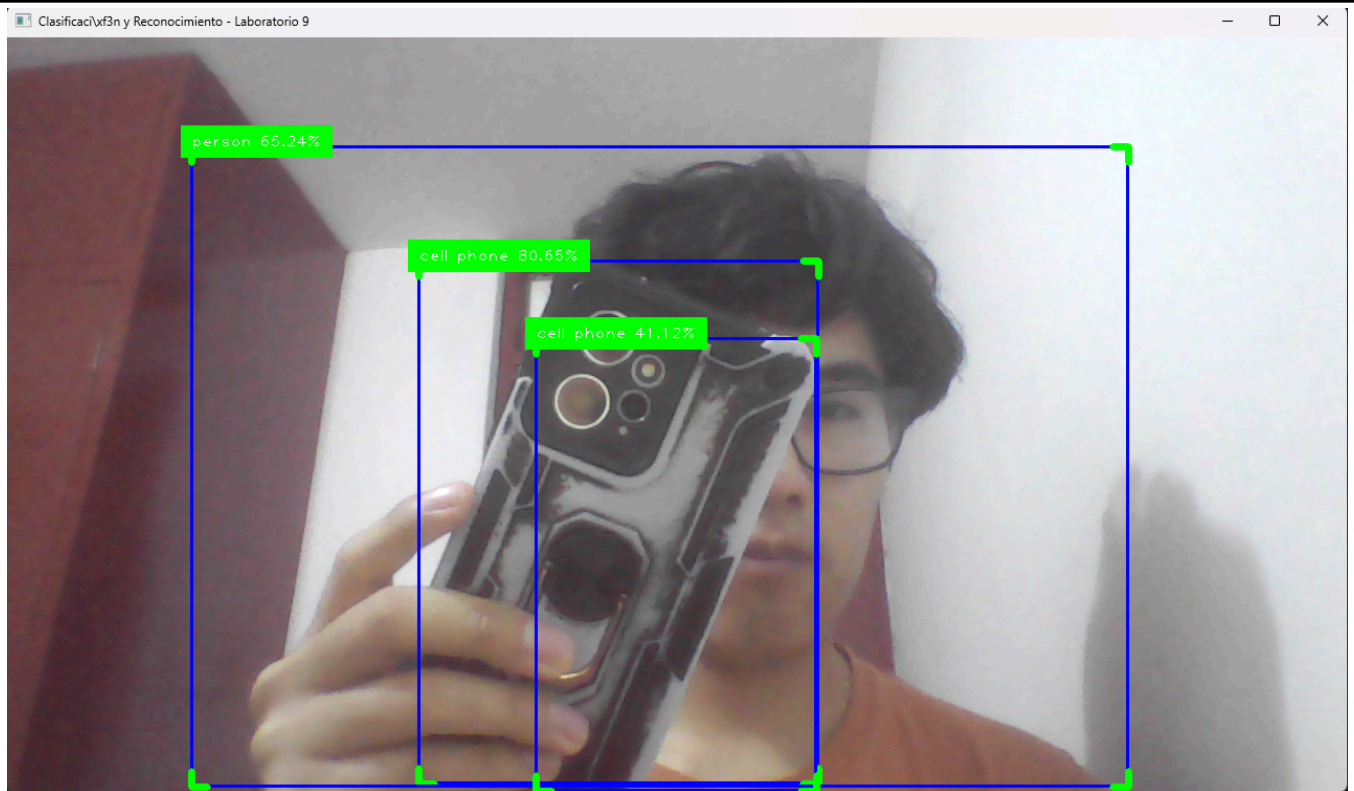
# Despliegue de la ventana interactiva en el sistema operativo
cv2.imshow("Camara en Tiempo Real - YOLOv8", img)

# Captura de interrupción por teclado: finaliza si el usuario presiona la tecla 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Liberación de recursos físicos de hardware ocupados por el programa
cap.release()
cv2.destroyAllWindows()
```

Ejecucion

python .\visioncomputacionalcamara.py



4.2. Script 2: Interfaz Gráfica Web Interactiva (Streamlit)

Python

```
import streamlit as st
from ultralytics import YOLO
import cv2
import numpy as np
from PIL import Image

# Configuración inicial del layout y metadatos de la página del navegador web
st.set_page_config(page_title="Detector de Objetos - Lab 9", layout="centered")

st.title("Clasificación y Reconocimiento de Objetos")
st.write("Suba una imagen para procesarla mediante la arquitectura de Inteligencia Artificial YOLOv8.")

# Optimización de carga: 'cache_resource' evita recargar el modelo de la memoria en cada interacción
@st.cache_resource
```

```
def load_model():
    return YOLO("yolov8n.pt")

model = load_model()

# 1. ADQUISICIÓN: Widget nativo de Streamlit que permite cargar archivos multimedia (JPG, PNG)
uploaded_file = st.file_uploader("Elige una imagen...", type=["jpg", "jpeg", "png"])

if uploaded_file is not None:
    # Lectura del archivo binario y transformación a formato de imagen estándar (PIL)
    source_image = Image.open(uploaded_file)
    st.image(source_image, caption="Imagen Original Subida", use_container_width=True)

    # Conversión obligatoria: PIL opera en canales RGB, pero OpenCV y YOLO interpretan matrices en BGR
    opencv_image = cv2.cvtColor(np.array(source_image), cv2.COLOR_RGB2BGR)

    # Evento detonador: Se ejecuta el bloque lógico solo tras presionar el botón interactivo
    if st.button("Ejecutar Reconocimiento"):
        with st.spinner("Procesando imagen con YOLOv8..."):

            # 2 y 3. SEGMENTACIÓN e INTERPRETACIÓN: Inferencia del modelo sobre la matriz de píxeles
            results = model(opencv_image)

            # El método nativo .plot() genera de forma automática una copia del frame con las cajas
            # y etiquetas correctamente posicionadas matemáticamente sobre la imagen.
            annotated_frame = results[0].plot()

            # Reconversión inversa de canales de color (BGR a RGB) para su correcta visualización web
            annotated_image_rgb = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)

            # Despliegue de los resultados procesados en la interfaz gráfica
            st.success("¡Procesamiento Completado con Éxito!")
            st.image(annotated_image_rgb, caption="Resultado del Análisis de Visión Computacional", use_container_width=True)

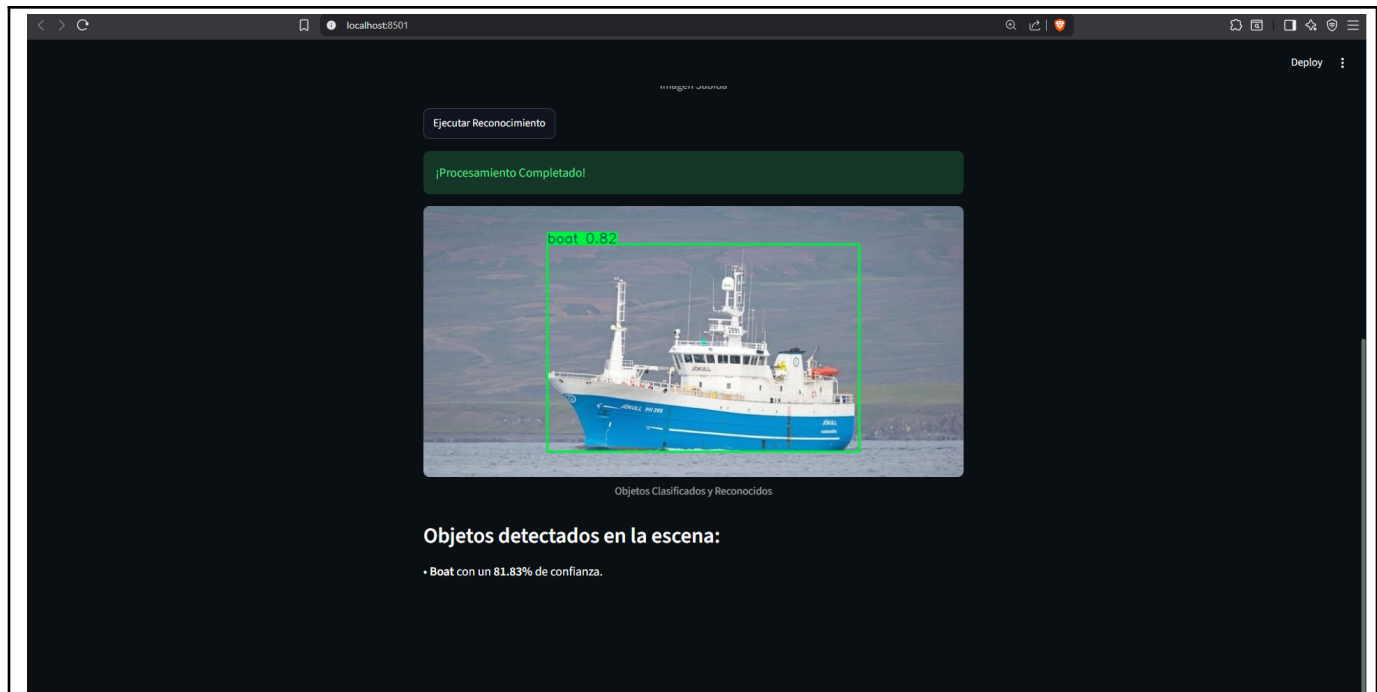
            # Desglose descriptivo en base de texto enriquecido (Markdown) para el usuario
            st.subheader("Listado de Objetos Reconocidos:")
            boxes = results[0].boxes
```

```
if len(boxes) == 0:
    st.write("No se detectó ningún objeto conocido en la base de conocimientos actual.")
else:
    for box in boxes:
        cls_id = int(box.cls[0])
        label = model.names[cls_id]
        confidence = float(box.conf[0]) * 100
        st.write(f"• **{label.capitalize()}** | Confianza del modelo: `{confidence:.2f}%`")
```

Ejecucion

streamlit run .\visioncomputacional.py







5. Análisis de Ventajas y Desventajas

Tecnología / Algoritmo	Ventajas Destacadas	Desventajas / Limitaciones
YOLOv8 (ultralytics)	• Velocidad de inferencia asombrosa en tiempo real en CPUs comerciales estándar. • Extrema precisión gracias a su diseño libre de cajas de anclaje (<i>anchor-free</i>). • Capacidad para discernir hasta 80 tipos de objetos concurrentes.	• El procesamiento interno opera bajo la lógica de "caja negra", lo cual complejiza la auditoría matemática exacta de los descriptores vectoriales intermedios.
Streamlit (Interfaz Web)	• Despliegue veloz y limpio de controles de usuario. • Manejo nativo de subida de	• Cierta retraso de latencia (<i>overhead</i>) al recargar por completo el script web ante

	archivos mediante drag-and-drop. • Excelente organización visual para la presentación de informes.	cambios de estados en la interfaz.
OpenCV + CVZone	• Control absoluto sobre los periféricos de captura analógica. • Simplificación drástica de líneas de código requeridas para dibujar textos geométricos adaptativos.	• El renderizado básico de OpenCV (cv2.imshow) carece de aceleración por hardware optimizada para entornos web embebidos.

6. Enlace al Video

 met-vcnp-ugj (2026-07-07 23:30 GMT-5)

7. Enlace a github: <https://github.com/MaxsForocca/LabComputacionGrafica>

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Describan un algoritmo utilizado para el reconocimiento y clasificación de objetos?

El algoritmo YOLO (You Only Look Once) es un claro exponente en este rubro. El núcleo funcional de este algoritmo se basa en el uso de Redes Neuronales Convolucionales (CNN) profundas estructuradas bajo un enfoque de paso único. A diferencia de técnicas heredadas (como las ventanas deslizantes o R-CNN) que segmentan la imagen en múltiples sub-regiones potenciales para evaluarlas de forma iterativa, YOLO procesa la matriz de la imagen completa en una única inferencia matricial forward. La red divide internamente la imagen en una cuadrícula de tamaño $S \times S$. Si el centro geométrico de un objeto se localiza en una celda, dicha celda asume la responsabilidad de la detección predictiva. De manera concurrente, la red calcula las coordenadas espaciales (x, y, w, h) correspondientes a las cajas delimitadoras, el

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

puntaje de confianza (confidence score) de que dicha caja enmarque un elemento real, y las probabilidades de pertenencia a las diferentes clases entrenadas del dataset.

2. ¿Qué problemas pueden ocurrir al realizar el reconocimiento y clasificación de objetos?

Al trasladar algoritmos de visión artificial al mundo real, surgen variadas problemáticas físico-matemáticas que alteran la señal visual analógica:

- **Oclusión Parcial:** Ocurre cuando un objeto en la escena es ocultado de forma física o interrumpido por la geometría de otro cuerpo intermedio. Esto trunca la extracción de descriptores visuales de contorno o textura esenciales, provocando fallos en la clasificación de la red.
- **Variabilidad de Iluminación:** Cambios abruptos en las fuentes de luz natural o artificial modifican severamente los valores discretos de brillo y contraste almacenados en los canales de color de los píxeles. Esto puede degradar la calidad matemática de los mapas de características convolucionales provocando omisiones en la detección.
- **Perspectiva, Escala y Rotación:** Si un objeto tridimensional es capturado por la cámara desde un ángulo atípico, la proyección bidimensional resultante altera las proporciones geométricas de las características que el modelo memorizó en su base de conocimientos durante su fase de entrenamiento original.

III. CONCLUSIONES

En conclusión, se implementó con éxito una solución moderna para la clasificación y reconocimiento de objetos utilizando Python y la arquitectura de vanguardia YOLOv8. Mediante el desarrollo de dos entornos complementarios —uno optimizado para el análisis dinámico en tiempo real mediante cámara web y otro estructurado como un panel interactivo web con Streamlit— se logró cubrir eficazmente cada etapa del procesamiento digital de imágenes, desde la adquisición y preprocesamiento de la señal digital hasta la segmentación y la interpretación automatizada basada en redes neuronales profundas.

Este enfoque no solo cumplió con las competencias de la asignatura orientadas al análisis y creación de aplicaciones multimedia, sino que también aportó un valor al escenario virtual exigido por la rúbrica, el laboratorio demostró cómo la inteligencia artificial optimiza y agiliza las tareas tradicionales de visión computacional, garantizando una herramienta interactiva, escalable y con un alto nivel de precisión técnica ideal para entornos académicos y comerciales.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13



REFERENCIAS Y BIBLIOGRAFÍA

- [1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8 Docs*. <https://docs.ultralytics.com>
- [2] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*. Stylus Publishing, LLC.
- [3] Streamlit Inc. (2026). *Streamlit Open-Source Documentation*. <https://docs.streamlit.io>
- [4] Mujtaba, R. Yolo V8: A Deep Dive Into Its Advanced Functions and New Features. <https://medium.com/@mujtabaraza194/yolo-v8-a-deep-dive-into-its-advanced-functions-and-new-features-f008599fe604>